



SORBONNE UNIVERSITÉ
Master AI2D

Teaching a Swarm to Understand: Gesture-Based Interaction with Robots

Internship Report - July 2026

Lyna HOCINE – Caroline ZOULOUMIAN

Supervised by Gilles Bailly and Nicolas Bredeche

Contents

1	Introduction	3
2	Project Context and Problem Definition	3
3	State of the Art and Existing System	4
3.1	Swarm Robotics and Collective Motion	4
3.2	Human-Swarm Interaction	4
3.3	Gesture-based Interaction and Simulation	5
3.4	Synthesis and Research Gap	5
4	Interaction Design and Concept Development	6
4.1	Brainstorming Process	6
4.2	Proposed Interaction Concepts	6
4.3	Octon Puzzle	7
5	Gesture Vocabulary	8
5.1	Interaction States	8
5.2	Selection and Command Mechanisms	8
5.3	Design Choices	8
6	Technical Approach	9
6.1	Simulator Architecture	9
6.1.1	Objectives and Design Choices	9
6.1.2	From the Original to the Developed Simulator	9
6.1.3	Simulation Model	9
6.1.4	C++ Runtime and Hardware Abstraction	10
6.1.5	Interaction and Gesture Pipeline	10
7	Collective Behaviours and Robot Dispositions	11
7.1	Formations in the Original Simulator	11
7.2	Formations in the Developed Simulator	12
7.3	Experiments on Octon Alpha Robots	12
7.4	Exploration of Swarm Motion Models	12
8	Gesture Recognition	13
8.1	Gesture Representation	13
8.2	Recognition pipeline	13
8.3	Implementation in the Simulator	13
8.4	Contextual Interpretation	14
9	Bravo Hand-Sensor Characterization	14
9.1	Experimental Objectives	14
9.2	Experimental Setup	14
9.3	Direction experiment	15
9.4	Height and Distance Experiment	16
9.5	Implications of Gesture Design	16
10	Results and Validation	17
10.1	Simulator Validation	17
10.2	Collective Behavior Results	17
10.3	Gesture Recognition results	18
10.4	Bravo Sensor Results	18

11 Discussion and Limitations	18
11.1 Interpretation of Results	18
11.2 Gesture Feasibility	19
11.3 Gesture Recognition Limitations	19
11.4 Decentralized Interpretation	20
11.5 Simulator Limitations	20
11.6 Physical Platform Limitations	20
12 Future Work	21
12.1 Deployment on Physical Robots	21
12.2 Integration of the real Robot Firmware	21
12.3 Continuation of the Hand Sensor Experiment	21
12.4 Improved Infrared Sensor Model	21
12.5 More Robust Gesture Recognition	21
12.6 Multi-user Interaction	21
12.7 Extension of the Octon Puzzle Concept	22
13 Conclusion	22

1 Introduction

Swarm robotics studies how multiple robots can coordinate to perform collective tasks like search and rescue, environmental monitoring, without relying on centralized control[1].

This decentralization can provide scalability and robustness, while individual robot control, such as a human operator issuing commands to each robot, does not scale well to large swarms[2].

In swarm robotics, robots typically rely on local perception rather than on having a global view of the system[1]. Communication is also typically limited to nearby neighbors rather than to the whole swarm, and collective behaviors emerge from local interactions rather than being dictated by a central controller. As a result, the collective behavior observed by the user does not result from individually specified actions, but emerges from the combination of local actions and interactions between robots.

Given these limitations, allowing a human to guide the swarm directly rather than programming every behavior becomes an attractive complement to autonomous control. But it is harder than it looks: a gesture that feels natural and effortless to a human, such as a swipe, must first be detected from the robots' local observations, then interpreted according to the context, and finally translated into a coordinated collective response.

Critically, the same gesture can mean different things depending on the context: which robots are currently selected, the current interaction state, the position and trajectory of the hand, and whether one or several users are interacting with the swarm at once.

These considerations motivate the use of interaction methods that feel natural to the user while remaining compatible with the swarm's decentralized, locally-perceiving nature. Hand gestures are a promising candidate because they provide a direct spatial interaction modality and can, in principle, be perceived locally by nearby robots rather than through a central camera.

A user might point toward a subgroup to select it, sweep a hand to indicate a direction of movement, open the palm to halt nearby robots, or trace a shape to request a specific formation. Yet recognizing the gesture itself is only part of the problem: the system must also determine which robots the gesture concerns, making targeting an important aspect of the interaction in addition to gesture recognition.

It is in this context that we pose our central research question: **how can natural hand gestures be interpreted and translated into coherent, coordinated, collective behaviors in a decentralized robot swarm?**

This work makes several contributions. First, we design and implement a human-swarm interaction simulator that models local robot perception, gesture recognition, and flocking dynamics without requiring physical hardware. Second, we conduct quantitative experiments characterizing the behavior of the Bravo capacitive sensor, informing how gesture signals should be modeled and processed. Third, we investigate what vocabulary of hand gestures is feasible and meaningful for swarm control, and how gesture meaning should depend on context and robot selection.

2 Project Context and Problem Definition

The Octon project is currently being developed by the doctorate student Thomas Juldo within the framework of a thesis at Ecole Supérieure d'Ingénieurs Léonard de Vinci's Institute for Future Technology and Sorbonne Université's Institute of Intelligent Systems and Robotics.



Figure 1: Octon Alpha Swarm

The development of Octon robots situates itself in the intersection of human-computer interaction and swarm robotics research. The goal is to create a swarm with which it is possible to interact directly by hand, through sensors, without the help of an overhead projector, which was the extent of previous work that inspired this one [3].

Different versions of the Octons are developed and in development. In this project, we worked with Octon Alpha, the original version (shown in Fig.1),

which uses eight infrared sensors to detect and interact with each other as well as rudimentarily detect obstacles. We then had access to the Octon Bravo’s (the latest version) hand sensor, which, placed on top of each robots, enables interaction and gesture perception.

To facilitate gesture interaction means working with the system’s constraints. Each robot has a local perception only and limited communication, which impacts decision making for the swarm. There can be sensor noise as well as occlusion.

In this project, we aim to create a simulated environment in which we can test the swarm interaction techniques, to investigate which hand gestures can be detected using the provided sensors, and to translate these gestures into collective robot behaviors. We will afterwards evaluate the system and identify the remaining challenges before real-world deployment.

3 State of the Art and Existing System

3.1 Swarm Robotics and Collective Motion

Swarm robotics studies how multiple robots can coordinate to produce collective behaviors through local interactions rather than relying on a centralized controller[1]. Each robot typically operates using information obtained from its local environment and from its neighbors, while the behavior of the swarm emerges from the interaction between individual robots. This decentralized approach allows collective behaviors to be maintained without requiring a global plan for the entire swarm.

Several approaches have been proposed to design such collective behaviors. They can be manually specified using behavioral rules or state-based models, or automatically generated using approaches such as evolutionary robotics and reinforcement learning. For collective motion, Reynolds’ Boids model is a widely used example of a rule-based approach. It showed that flocking behavior can emerge from three local rules: separation, alignment and cohesion [4].

Similarly, the Vicsek model describes collective motion through local alignment between neighboring agents and demonstrates how ordered group motion can emerge from simple local interactions [5]. Turgut et al. demonstrated the application of flocking rules to physical robots using local sensing, showing that coordinated motion can be achieved without global information [6].

Beyond general flocking, swarm robots can also be coordinated to maintain specific geometric formations. Formation control can be implemented using approaches such as leader-follower strategies, where some robots determine the group’s motion while others maintain relative positions, or decentralized behavior-based approaches in which formation keeping is combined with local behaviors such as obstacle avoidance. Formations may also be static or dynamically reconfigured, allowing the shape of the swarm to change according to the task or user input. These approaches are particularly relevant to this work, where collective motion and formation control are used as the basis for translating user gestures into swarm behaviors.

3.2 Human-Swarm Interaction

The decentralized nature of swarm robotics raises another question: how can a human influence collective behavior without having to control individual robots? Human-swarm interaction (HSI) studies this problem, ranging from direct control of individual robots to methods that influence the behavior of the swarm as a whole. Kolling et al. [2] distinguish between global and local interaction approaches. In global interaction, a command sent by an operator can reach the entire swarm, whereas local interaction targets only robots in a specific part of the environment, allowing different parts of the swarm to receive different instructions.

Gestures provide one possible way of expressing such commands. Podevijn et al. [3] demonstrated gesture-based control of a swarm using a Kinect sensor, showing that a small vocabulary of gestures, including selection, splitting, merging and rotation, can be used to control both the swarm as a whole and selected groups of robots. This highlights an important aspect of swarm interaction: interpreting a command also requires determining which robots it applies to. Selection can be based on spatial proximity, an explicit selection gesture, or another interaction mechanism.

For a decentralized swarm, this interpretation can itself be distributed across the robots. Giusti et al. [7] demonstrated a distributed gesture-recognition approach in which each robot makes a local classification and

the swarm reaches a common decision through multi-hop voting. This approach illustrates how perception and interpretation can be integrated into the decentralized architecture of the swarm, rather than relying on a central recognition system.

3.3 Gesture-based Interaction and Simulation

The work discussed above shows that gestures can provide a direct way for users to express commands to a swarm, but this requires the systems to reliably recognize the gesture being performed. A variety of approaches have been proposed for gesture recognition, including neural networks, statistical classifiers and hidden Markov models. The $\$$ -family of recognizers provides a simpler alternative based on gesture trajectories. In particular, the $\$1$ Unistroke Recognizer was designed as a lightweight recognizer that can be implemented with relatively little code while providing competitive recognition performance on the gesture sets evaluated in its original study [8]. The $\$P$ recognizer extends this family to point-cloud gestures, providing an alternative when the order of points in the trajectory is less important [9].

For this work, the $\$1$ recognizer is particularly relevant because of its simplicity and its use of a 2D gesture trajectory. The recognizer normalizes an input trajectory before comparing it with stored templates to determine the closest match [8]. Extending trajectory recognition to three dimensions introduces additional complexity. The $\$3$ recognizer directly extends the $\$$ -family to 3D gestures by considering candidate rotations when matching a gesture against templates [10]. Alternatively, a 3D gesture can be projected onto a 2D plane and recognized using $\$1$, although this discards depth information. These considerations are relevant when designing gesture interaction for robots with spatially distributed sensing.

Gesture-based interaction has also been explored in systems where the robots themselves form the interface. Le Goc et al. introduced Zooids, a platform of small robots designed for tabletop swarm user interfaces [11]. In this system, users can interact directly with the robots while the swarm can simultaneously represent information or respond through physical motion. Zooids therefore demonstrates how a swarm can act as an interaction medium rather than simply being controlled through an external interface. However, its tracking and control architecture relies on centralized infrastructure, whereas the system considered in this work is designed around local robot perception.

Developing such decentralized interactions on physical robots can be costly and difficult to reproduce. Simulation therefore provides a useful environment for developing and evaluating swarm behaviors before deployment on hardware. It allows experiments to be repeated under controlled conditions and provides direct access to robot states, sensor readings, and communication. At the same time, simulated sensing, communication and physical dynamics are approximations of their real-world counterparts, creating a reality gap that must be considered when transferring a system to physical robots.

3.4 Synthesis and Research Gap

The literature reviewed above provides several building blocks for human interaction with robot swarms. Decentralized collective motion and formation control allow groups of robots to produce coordinated behaviors from local interactions, while human-swarm interaction research has explored ways for users to influence these behaviors through global, local and gesture-based commands. Gesture recognition methods provide a way to translate user movements into discrete commands, and interactive swarm platforms demonstrate the potential of using the robots themselves as an interaction medium.

However, these aspects are generally studied separately. In particular, there is limited work combining local perception, gesture recognition, gesture targeting and context-dependent interpretation with decentralized collective behavior in a single interaction system. This is the focus of our work. We develop a simulator in which a user can interact with a swarm through hand gestures, while the robots operate according to local perception and collective motion rules. This provides a controlled environment for exploring how gestures can be translated into swarm-level behaviors while remaining compatible with a decentralized architecture.

4 Interaction Design and Concept Development

4.1 Brainstorming Process

To identify a gesture vocabulary that would be both intuitive for the user and feasible to detect, we began by brainstorming the range of possible input gestures and their corresponding output behaviours for the swarm. Sketching was used throughout this process, both to generate ideas quickly and to communicate interaction concepts to each other and to our supervisor.

Two main criteria guided the selection of gestures. First, a gesture had to be detectable with the sensing hardware available to us, an idea that looked promising on paper but could not be reliably measured by our sensors was discarded regardless of how natural it felt. Second, a gesture had to be discoverable meaning a user should be able to guess, or quickly learn, its meaning without extensive prior explanation, favoring gestures that build on familiar, everyday hand movements[3], [12].

Several gesture concepts were considered and later rejected under these criteria. A "whooshing" come and go motion, intended to feel like beckoning or waving robots away, was discarded for being too similar to the swipe gesture while adding no additional detectable information, since our sensor could not distinguish direction. Multi-hand and multi-user gestures were also considered, but this direction was not pursued further within the scope of this project, as it would have required testing and validation beyond what our timeline allowed.

The gestures were then progressively narrowed down as we learned more about the constraints of each sensor. In particular, discovering that the hand sensor could only measure distance, not direction, forced us to reconsider the swipe gesture: rather than recognizing the direction of a swipe, as originally envisioned, we redefined the gesture around what the sensor could actually detect while preserving the swipe interaction.

4.2 Proposed Interaction Concepts



Figure 2: Post-its sorted on an axis from gamified concept to more artistic concepts

We considered different concepts for the final product, and provide a brief description of each. Fig.2 shows a part of our brainstorming process, using post-its.

Octon Dance Orchestra: The idea here is to control the "swarm dance" by having one or two users act like orchestra conductors. Different gestures can change the movement, setup, speed... of the swarm.

Hunting game: A two-player game in which one player selects the robots individually to capture them by stopping them, and the other player manipulates the remaining swarm to escape. The touched robots cannot be manipulated as they are captured. Some additions could be a mechanism of liberation of the captured robots, and different gestures of dispersion and guiding for more complexity.

Checkers: A two-player game in which the goal is to "eat" all the other team's robots.

Capture the flag: A two-player game in which each team tries to capture a zone/flag in the arena. Each team has a different led color. Gestures enable each team to attack, defend, regroup, separate the swarm.

Octon Beatbox: One or more users can interact with the swarm via different gestures to create a musical piece. Grouping different robots together and putting them in different setups can produce different rhythms and sounds.

Snake game: Just like in the original snake game, the user must guide the aligned Octon Snake and avoid hitting obstacles or itself, while collecting more robots.

Fish Game: Similar to the snake game, can be played by one or more users. Each robot cluster can be guided and absorb smaller clusters. The goal is to absorb all the other robots.

Swarm creature: More artistic, this idea consists in manipulating the swarm as a creature, giving it reactions that seem more biological.

Coding the swarm: The user "programs" the swarm using gestures one after the other, and then the swarm recreates the moves together. The original idea comes from the serious game SPY (by LIP6 Mocah team) in which the user programs robots in advance using blocks to attain certain states. This mechanism can be found in some other games as well, such as the board games Colt Express and Ricochet Robots. It is a different way to give an introduction to algorithms for beginners.

Ping pong: A two-player game in which the users send the swarm to each other.

Flitzpuck: A swarm version of the game of Flitzpuck, originally a wooden two player game in which each player tries to send all of the pucks to the other side and to be the first one with a cleared area. Here, the game could be enriched with different swarm mechanisms provoked by gestures, to block other robots or to send more at the same time to the other side. For example, we could make a bigger group at the expense or becoming slower, which would enable each player to decide on which strategy they would like to adopt.

Octon Puzzle: A solo or multiplayer brain-teaser game in which the players have to find ways to arrange the robots in different ways. It is the proposition that we ended up choosing as our final prototype.

4.3 Octon Puzzle

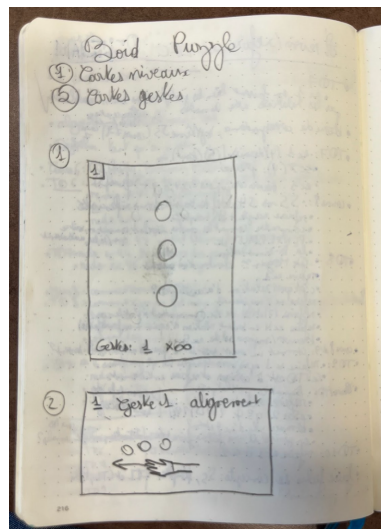


Figure 3: The puzzle game sketch (using its old name, "Boid Puzzle")

In the Octon Puzzle game (Fig.3), the goal for the player(s) is to arrange the robots in the formation described by a given card.

Cards with a drawing of the expected formations are given to the player(s), with an increasing level of difficulty, as well as a gesture dictionary. We opted for the idea of paper / cardboard cards, to make it feel like a board game.

To increase the challenge, the players could unlock new gestures with new levels, enabling more and more complex formations. There could also be a limit for the amount of times a gesture could be used, but that would also imply that the start formation would be specified as well. This could be implemented different ways: the start formation could be the end formation of the previous level. It could also be the same for each level, and be implemented by the robots at a "restart command", which could be a gesture or a voice command. A voice command could also be used to order the robots to a specific formation for each other: the command "start level 1" would trigger a different formation than "start level 5".

The led color function could be used for more challenge as well, by adding levels in which robots of different colors should be positioned in specific places.

If multiplayer, each player could have a limited gesture dictionary (half of the dictionary each for example), to make it a collaborative experience. Formations do not have to be static.

To check the correctness of a formation, vocal recognition could be

used to indicate the level.

This idea stayed at the prototype level, and would need an update as the possibilities changed regarding the capabilities of the sensors.

5 Gesture Vocabulary

The gesture vocabulary was designed around a simple principle: a gesture cannot be interpreted independently of the interaction in which it occurs. The same movement can have different meanings depending on which robots are involved, whether a selection has already been made, and what the system is currently expecting from the user. We therefore first define the interaction states that provide this context, before describing how selection and commands are handled[3], [12].

5.1 Interaction States

The meaning of a gesture depends on the current state of the interaction. For example, a swipe performed when no robot has been selected cannot have the same meaning as a swipe performed after a group has been selected. We therefore use interaction states to provide the context needed to interpret each gesture.

The interaction can be described through several stages: *idle*, *detecting user*, *selecting robots*, *receiving commands*, *executing behavior*, and *providing feedback*. In the *idle* state, the robots continue their default behavior until a hand is detected. When a hand enters the sensing range, the system can begin observing it without immediately interpreting its movement as a command.

Selection then establishes which robots the user’s subsequent actions should concern. A robot approached by the hand is first considered as a candidate and must be explicitly confirmed before becoming selected. Once one or more robots have been selected, subsequent gestures can be interpreted as commands for this group.

5.2 Selection and Command Mechanisms

In the simulator, robot selection is represented through three states: **IDLE**, **CANDIDATE** and **ACTIVE**. A robot becomes a candidate when the hand hovers over it, and a confirmation changes its state to **ACTIVE**. Confirming an already active robot cancels its selection.

The `circle_select` gesture provides this confirmation by selecting the robots currently being hovered over. This creates a distinction between approaching a robot, which identifies a possible target and deliberately selecting it.

The physical prototype follows a similar principle. The capacitive signal is first calibrated and monitored for an approaching hand. A sufficiently long stable interaction can trigger selection, while shorter interactions can be used to detect other gesture events.

In the current simulator, however, the mapping from gestures to swarm behaviors is only partially implemented. The selection gesture is connected to an actual action, while directional swipes and the *V*-shaped formation gesture are currently recognized and logged but do not yet directly trigger robot behaviors.

5.3 Design Choices

The gesture vocabulary was designed by considering the capabilities of the available sensors, observations from the literature, the behavior of the simulator, ease of execution and the decentralized nature of the swarm.

The Bravo sensor experiments were particularly important in narrowing the vocabulary. Since the sensor did not provide enough information to reliably determine the horizontal direction of a gesture, directional gestures cannot currently be considered reliable commands when inferred from local sensor measurements alone. Instead, the interaction can rely on properties that are more directly observable, such as hand presence, proximity and dwell time.

At the same time, trajectory-based recognition allows more expressive gestures to be represented when a sufficiently informative trajectory is available. The final vocabulary therefore combines simple proximity-based interactions with trajectory-based gestures, while remaining small enough to limit ambiguity and keep the interaction understandable for the user.

Overall, gesture design was treated as a compromise between what is intuitive for the user, what can be sensed reliably by the robots, and what can be interpreted within a decentralized swarm.

6 Technical Approach

6.1 Simulator Architecture

6.1.1 Objectives and Design Choices

The simulator was developed to provide us with a controlled environment for designing and evaluating human-swarm interaction before deploying the code to physical robots. At the time of the internship, access to physical robots was limited, making simulation necessary for developing and testing the proposed interaction system. The simulator was therefore designed to reproduce the main characteristics of the physical platform while allowing experiments to be conducted quickly and repeatedly.

A first design objective was to preserve the decentralized nature of the swarm. Each simulated robot should operate using only information that would be available to its physical counterpart, such as local sensor measurements and communication with neighbors. Global information about the state of the swarm is therefore not provided to the robot control logic. This constraint is particularly important for evaluating whether collective behaviors emerge from local interactions rather than from centralized control.

Our second objective was to minimize the difference between the code tested in simulation and the code intended for the physical robots. The simulator therefore executes the robot logic directly in C++ through a simulated hardware abstraction layer. This layer provides simulated access to sensors, actuators and communication while remaining an interface compatible with the physical robot firmware. Together, these design choices allow swarm behaviors and interactions to be developed in simulations while remaining close to their eventual hardware implementation.

6.1.2 From the Original to the Developed Simulator

This internship started from an existing simulator designed to develop and test swarm behaviors. While it provided the basic environment and robot simulation required for collective motion, it was not designed for human-swarm interaction through hand gestures. In particular, it did not represent the user’s hand or record hand trajectories, which prevented the development of gesture-based interactions.

The original simulator also differed from the intended physical execution environment. Although the robot behaviors were written in C++, the simulator translated the logic into TypeScript before execution. Consequently, the code tested in simulation was not exactly the code that would run on the physical robots. This made it desirable to execute the C++ code directly and provide it with simulated equivalents of the hardware interfaces.

These limitations led to three main requirements in our simulator: support for a simulated human hand and gesture trajectories, direct execution of the robot’s C++ control code, and a simulation architecture compatible with the local sensing and communication constraints of the physical swarm. The resulting simulator extends the existing environment while introducing these capabilities. Its main components are described in the following sections.

6.1.3 Simulation Model

Robot Motion The simulated robots use a differential-drive model, in which the motion of the robot is determined by the velocities of its two wheels. Given the left and right wheel velocities v_L and v_R , the linear and angular velocities of the robot are computed as

$$v = \frac{v_R + v_L}{2} \tag{1}$$

$$\omega = \frac{v_R - v_L}{d}, \tag{2}$$

where d is the distance between the two wheels. The robot pose is then updated according to its current orientation θ :

$$x_{t+\Delta t} = x_t + v \cos(\theta_t) \Delta t \tag{3}$$

$$y_{t+\Delta t} = y_t + v \sin(\theta_t) \Delta t \tag{4}$$

$$\theta_{t+\Delta t} = \theta_t + \omega \Delta t. \quad (5)$$

The simulator also models collisions between robots. When two robots overlap, their positions are corrected so they no longer intersect, while preserving their simulated motion. This provides a simple approximation of physical collision handling without requiring a full physics engine.

Perception The simulator models the local perception available to each robot through infrared (IR) sensors. Each sensor is represented by a detection cone defined by its position, orientation, angular range, and maximum detection distance. The sensors are used to detect nearby robots and obstacles, providing each robot with local information about its surroundings.

For each simulated sensor, potential objects are identified according to their distance and relative angle. The distance to a detected object is then converted into a normalized sensor value based on the sensor’s maximum range. This provides the robot control code with a representation of its local observations without exposing the global state of the simulation.

The resulting sensor readings are used by the robot behaviors to react to nearby robots and obstacles. This preserves the local perception constraint of the physical swarm within the simulator: each robot can access its own sensor observations, but does not directly access the positions or states of robots outside its sensing range.

Communication Inter-robot communication is simulated through a virtual network that allows robots to exchange messages with one another. Each robot can send and receive messages through the same communication abstraction used by the robot control code, while the simulator handles their transmission between the simulated robots.

The communication model is designed to preserve the decentralized organization of the swarm. Robots do not access a shared representation of the swarm state through the network, instead, information must be explicitly exchanged between robots. This allows communication-based swarm behaviors to be tested in simulation while remaining consistent with the communication model expected by the physical robots.

The virtual network therefore acts as an intermediate layer between the robot control code and the simulator. The implementation also provides mechanisms for synchronizing communication with the simulation loop, ensuring that messages are processed as part of the robot execution cycle.

6.1.4 C++ Runtime and Hardware Abstraction

A central design choice of the developed simulator was to execute the robot control code directly in C++. This avoids the discrepancy present in the original simulator, where C++ code was translated into TypeScript before being executed. The developed simulator instead provides a simulated hardware abstraction layer (HAL) that allows the same control code to interact with simulated sensors, actuators and communication interfaces.

The simulated HAL reproduces the main hardware interfaces required by the robot firmware. From the perspective of the control code, operations such as reading sensor information, controlling the motors and communicating with other robots are accessed through the same type of interfaces on the physical platform. The simulator implements these interfaces internally and connects them to the corresponding simulated components.

This architecture separates the robot control logic from the simulation itself. The control code does not need to know whether it is interacting with simulated or physical hardware, it only accesses the interfaces provided by the HAL. This makes the simulator closer to the intended deployment architecture and allows the same C++ behaviors to be developed and tested before being executed on the physical robots.

6.1.5 Interaction and Gesture Pipeline

The interaction system is built around the user’s hand as an input to the swarm. A simulated hand can be positioned in the environment and moved to produce gesture trajectories. The robots perceive the hand through their local sensors, allowing the gesture to be processed within the same local perception framework as the other elements of the simulation.

The recorded hand trajectory is passed to the \$1 Unistroke Recognizer[8], which compares it with a set of predefined gesture templates and returns the best-matching gesture. The recognized gesture is then

interpreted according to the current interaction context, including the robots concerned by the interaction and the current state of the system.

The resulting command is passed to the robot control layer, where it can trigger an individual or collective behavior. This separates the interaction into three stages: gesture perception and recognition, interpretation of the gesture in the context, and the execution of the corresponding swarm behavior. The simulator therefore makes it possible to evaluate the complete interaction pipeline, from the user's hand movement to the resulting collective behavior.

7 Collective Behaviours and Robot Dispositions

Alongside the gesture-based interaction, we explored several collective behaviors for the Octon robots. These experiments covered predefined formations, obstacle avoidance, and flocking models. They were conducted both in simulation and on the physical robots, depending on the behavior being investigated.

7.1 Formations in the Original Simulator

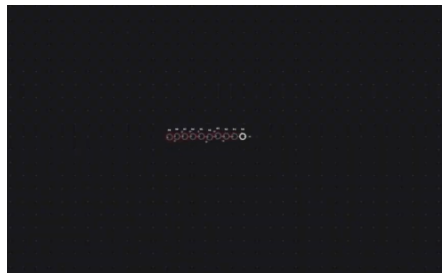


Figure 4: Chain

Using the simulator developed by Thomas Juldo, we implemented different Octon formations. At this moment in time, these versions only work on this simulator. These versions are all leader based, with a leader assigned based on robot index: the robot with the lowest index is the leader. In improved setups, the leader should be designated based on position, with a voting system. Each robot get its neighbors' bearing and range through `robot->get_polar_vector_to_robot()` and moves toward a target distance and heading with `robot->differential_drive->set_target()`. Obstacle avoidance is also used. We detail each formation tried in this simulator.

Chain: Each follower tracks the robot with the previous index (index i follows index $i - 1$). When the measured distance with the previous robot is too high, it moves towards it. When the measured distance is too low, it stops. We implemented two variants of the chain formation, differing in how the leader (the robot with index 0) behaved. In the first version, the chain is static: the leader does not move, and the other robots move to form a chain which quickly becomes static. In the second version, the leader moves continuously at a fixed speed, which implies that the chain moves as well. The result is shown in Fig.4.

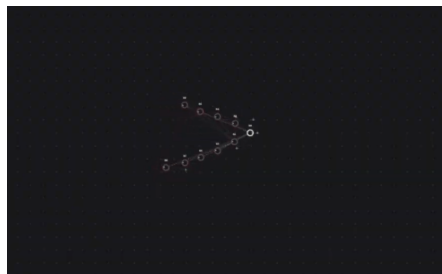


Figure 5: V formation

V formation: This formation is an extension of the chain-following method, but splitting the followers into two branches. Here, we separate them based on robot index: odd indexes follow each other, and the same goes for even indexes. An angular offset is applied to both sides, so that they are separate. The leader moves at a constant forward speed. The desired following distance is higher than for the chain, in order to prevent the two branches from colliding. The result is shown in Fig.5.

Line: Our attempt for alignment is based on the angle between each robots. The goal is for all the robots to face the same way, with an angle of 180° between robots. The current implementation did not fully accomplish alignment in a straight line. Like in the chain implementation, each follower tracks the previous robot for distance. However, it also computes a heading needed to keep the line straight. This heading is calculated from the leader's bearing, offset by 180° .

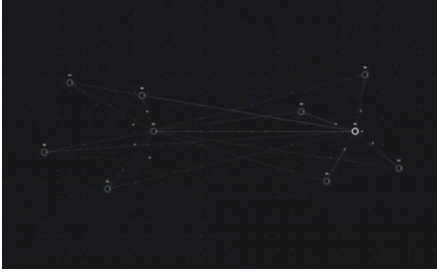


Figure 6: Separation

Separation: The separation formation implements a hand triggered separation, using the simulated IR readings. The first robot to detect the hand flees directly away from it. Every other robot determines which side it is on and moves to the opposite direction from the other group. Robots within a fixed radius from where the hand was perceived participate in the movement, and the further away they are the weaker their response is. The result is shown in Fig.6.

7.2 Formations in the Developed Simulator

The formations implemented in the original simulator were used as a starting point for the developed simulator. They provided a basis for testing whether the existing collective behaviors could be reproduced

within the new simulation architecture while preserving the local perception and communication mechanisms. The developed simulator was then used to explore these behaviors together with the hand interaction and gesture recognition pipeline. This made it possible to test the collective behaviors under controlled conditions while varying the position and movement of the hand.

7.3 Experiments on Octon Alpha Robots

In contrary to the previous formations, which run on simulators, these experiments were run directly on the physical Octon Alpha robots. We tested different related behaviors: following the nearest obstacle, avoiding obstacles using physical collision detection, and avoiding obstacles using IR sensing.

Following the nearest obstacle: Here, the robot moves towards what its sensors report as the nearest obstacle. In order to detect obstacles (and not other robots), we call `get_polar_vector_to_robot()` with the robot's own ID. This doesn't distinguish between a robot and a hand, but allows the detection of the hand by self reflection. The robot faces the obstacle, and moves towards it, stopping when it is within a certain reach.

Collision based obstacle avoidance: This approach directly detects collisions from the IMU. The robot monitors its acceleration, and if it detects a spike above a fixed magnitude, it interprets it as a collision. It then turns away before continuing to move forward. We implement an escalation in the turns if they occur quickly together: a single collision produces a quarter turn, and more collision will increase the degree of the turn. There is a cooldown period as well, which avoids triggering the effect on the same impact. In situation where a small turn isn't enough, the robot will be able to get away quicker with the escalation.

IR based obstacle avoidance: To avoid obstacles using IR sensing, we read the raw IR reading directly, and compare it against a fixed threshold to determine if an obstacle is present. If there is an obstacle, the robot turns 90° and moves forward. We tested another approach, which detects obstacles directly through swarm observation: if a robot observes an object with its own ID in its `swarm_observations`, under a certain range, it means that the robot's emitted IR is being reflected. This, we can avoid the obstacle by turning.

7.4 Exploration of Swarm Motion Models

Along with the obstacle and formation specific behaviors, we explored Reynold's Boids [4]. We implemented a test of the three flocking rules, first on its own and then combined with the gesture vocabulary we explored.

Our implementation computes separation, cohesion and alignment using `robot->perceived_by_others[]`, the swarm's shared robot perception structure. Each robot's contribution is weighted by proximity, and added to a 2D vector for each rule. The vectors are then summed (with a forward speed bias), and converted into a command for each robot. For alignment, each robot finds



Figure 7: Flock implementation on the original simulator

the heading of each neighbors by comparing their perception to its perception, and goes towards the average computed direction. An implementation on the original simulator is shown in Fig.7.

We further use the flocking behavior with hand gestures. This lets the use tune the three rule weights. Holding the hand near a robot for one full second corresponds to the "select" gesture, which increases the separation weight and leads to a temporary speed increase. A constant passing of the hand (towards the robot, then away from it) is a "swipe" gesture, and increases the alignment weight. A "tap" gesture, which means two quick going near than going away movements, correspond to an increase of the cohesion weight.

8 Gesture Recognition

8.1 Gesture Representation

In this system, a gesture is represented as a trajectory: an ordered sequence of timestamped points describing the movement of the user's hand through space. Each point contains the hand's three-dimensional position and orientation at a given instant. Gesture recognition therefore operates on a spatial trajectory rather than on a single hand pose.

Two representations of the trajectory are maintained separately. The *real trace* corresponds to the ground-truth trajectory of the simulated hand and is recorded directly by the simulator. The *perceived trace* is reconstructed from the observations available to the robots. Since each robot only has access to local sensor measurements, the hand position is estimated by combining the individual robot observations into a single trajectory. Each robot's contribution is weighted according to its distance from the hand, giving greater influence to robots that are closer to it.

This distinction allows the effect of local perception to be evaluated separately from the gesture recognition process. The recognizer operates on the perceived trace, so differences between the real and perceived trajectories reflect the limitations of the swarm's sensing rather than the classification method itself.

8.2 Recognition pipeline

Gesture recognition begins when the user starts recording a trace. During recording, the system stores the real hand trajectory while simultaneously reconstructing the perceived trajectory from the robot observations described above. Once recording stops, the perceived observations are merged into a single trajectory using the distance-weighted softmax centroid. The resulting trajectory is converted to the point format required by the \$1 recognizer[8].

Traces containing fewer than eight points are rejected as too short to classify reliably. For valid traces, the trajectory is projected onto the horizontal plane and converted from meters to millimeters before being passed to the \$1 recognizer[8]. The recognizer compares the input trajectory with the registered gesture templates and returns the best-matching template together with a similarity score. A recognition result is accepted only when this score exceeds a predefined threshold, otherwise, the gesture is considered unrecognized.

A generic swipe is used to represent directional commands without requiring separate templates for each direction. Once the swipe shape has been recognized, its direction is determined from the real trace by comparing the horizontal and vertical displacement between its starting and ending points. This separation is deliberate: the experiments presented in Section 9 show that the available robot sensors do not provide sufficient information to reliably determine the direction of a gesture from local sensor readings alone. Direction is therefore currently obtained from the ground-truth hand trajectory in the simulator rather than reconstructed from the perceived trace. A single swipe template can consequently represent four directional commands while keeping the gesture vocabulary compact.

8.3 Implementation in the Simulator

The gesture recognition system is organized into three main components.

`DollarGestureRecognizer` interfaces with the underlying \$1 recognizer and manages the gesture templates used by the simulator.

`GestureTemplateStore` handles the persistence of user-defined templates using the browser’s local storage, allowing templates to remain available between sessions.

Finally, `HandGestureEngine` provides the interface used by the rest of the application to perform gesture recognition without depending directly on the implementation of these components.

The simulator also allows users to create and wave new gesture templates from recorded hand traces. A recorded trace can be assigned a name and added to the template collection, making it available for subsequent recognition. Templates can also be cleared through the interface.

After recognition, the resulting gesture name is mapped to an interaction action. Currently, this mapping is implemented for the `circle_select` gesture, which confirms the selection of robots currently under the hand.

8.4 Contextual Interpretation

Recognizing a gesture’s shape is not sufficient to determine the user’s intention. A recognized gesture does not, by itself, specify which robots it applies to or how it should be interpreted in the current interaction state. This information is provided by the interaction context, including the current selection and interaction state described in Section 5.1

In addition to the trajectory-based \$1 pipeline, the simulator contains a second gesture-detection mechanism based on the raw capacitive and infrared sensor signals of individual robots. Each robot monitors its sensor readings for sudden deviations from their recent baseline, which can indicate local events such as a hand approaching a particular side of the robot or the robot being touched, tilted, or picked up. A separate mechanism can also identify spatially and temporally correlated events across several robots and interpret them as a swarm-level gesture.

These two mechanisms operate independently. The trajectory-based pipeline recognizes gestures from the recorded hand trajectory, while the capacitive and infrared system interprets local sensor events directly. Consequently, the same physical hand movement can produce different observations in the two systems. At the current stage of the project, only the trajectory-based pipeline is connected to an actual robot action, through the `circle_select` gesture. The remaining recognized gestures and sensor-based events are logged as intended actions but do not yet modify the robot behavior.

9 Bravo Hand-Sensor Characterization

9.1 Experimental Objectives

We established an experimental procedure in order to characterize the perception of the Bravo hand sensor. Once given the sensor, distached from a full Octon Bravo robot, we needed to understand what the capacitive hand sensor could actually measure. This would then enable us to design a gesture vocabulary that would be more adapted. Indeed, it was previously assumed that the robot would be able to tell in which direction was the hand, but we hadn’t been able to evaluate the sensor at that point. Our experimental procedure enabled us to visualize the sensor signal in different directions, for example with polar maps such as Fig.8, which shows the values at height 0.

We designed the experiment to understand the range of object detection by the sensor, and if the direction of the object could be recovered from the sensor signal. This would then help us evaluate possible gestures.

9.2 Experimental Setup

We collected sensor readings with the logging script `serial_logger.py`. The script records the raw data output from

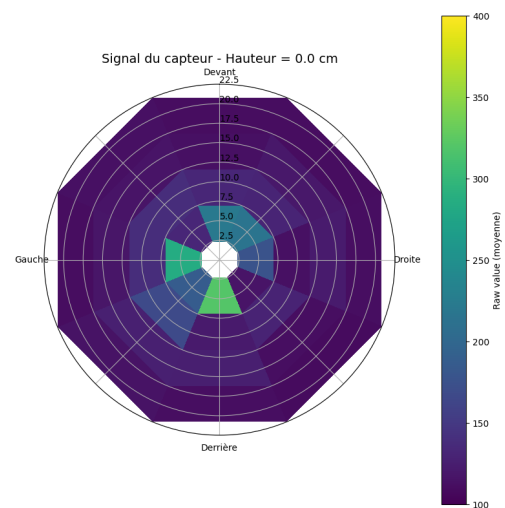


Figure 8: Polar map of our collected data at height 0.

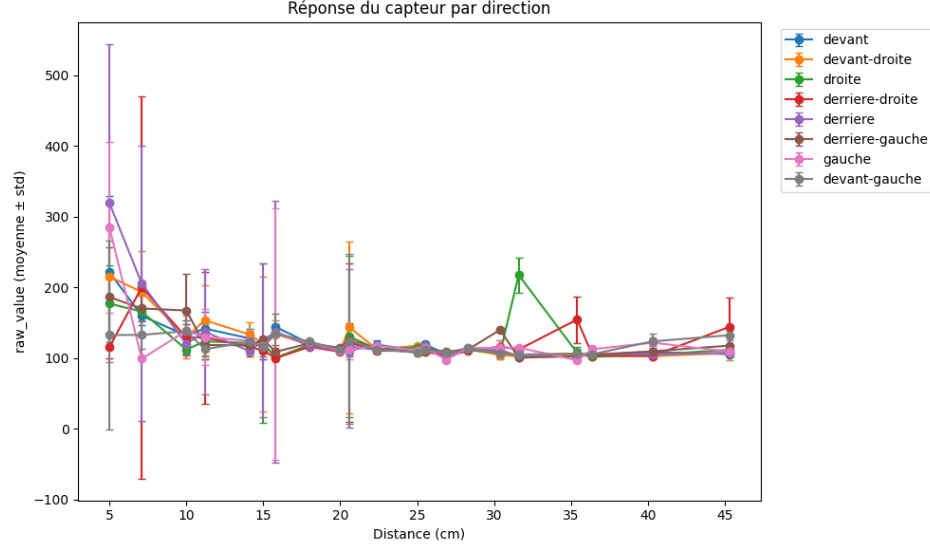
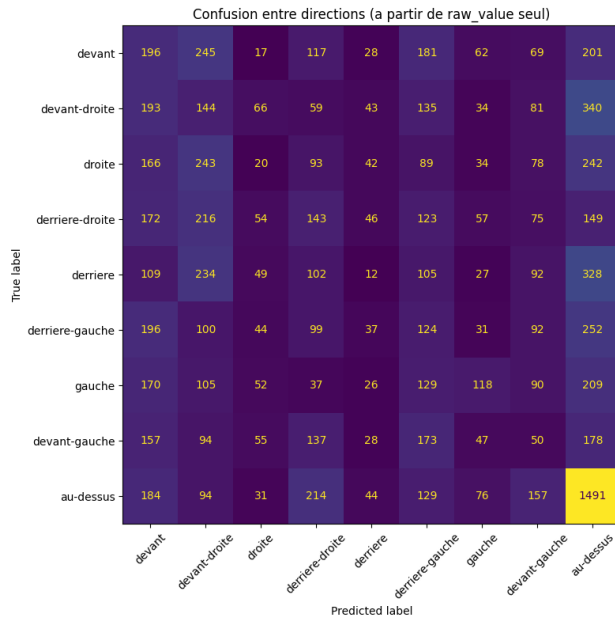


Figure 9: A plot of raw value against distance for each direction

the sensor, and adds it to a csv file at each timestamp, along horizontal direction (one of eight horizontal compass-style directions, devant, devant-droite, droite, derriere-droite, derriere, derriere-gauche, gauche, devant-gauche, plus a ninth au-dessus position directly above the sensor), reach and height above the sensor (both in cm) as well as the corresponding Euclidian distance, and a trial_id. Each position is captured for five seconds, and set and held by hand. The script provides a timer before the data is taken to enable positioning.

The csv file log, data/hand_sensor_data.csv, is large (722.6 Mo, 8,442,696 rows) due to the fact that the sensor was logged continuously, to enable us to look at variations in the data during a trial. However, since the signal is constant for most of a trial's duration, we unduplicated consecutive identical readings within each trial before analysis, which reduced the dataset to 10,561 representative rows spanning 230 unique trials.

9.3 Direction experiment



Before the experiment, we did not know whether the horizontal direction could be recovered from a reading. We decided to test this.

To do so, we recorded the raw sensor value for hand placed at the the eight directions and a range of distances, combining different heights and reaches, using the protocol described in the previous section.

When plotting the mean raw value against distance separately for each of the eight direction (as seen in Fig.9), we saw that the curves overlapped almost completely, without any conclusion that could be made to separate the directions at a specific distance basing ourselves only on the raw value. To test our now hypothesis that there we couldn't infer direction from raw value, using more rigorous methods, we decided to train a KNN classifier to predict direction from raw value alone. Our KNN reached 21.8% accuracy, which was lower than our baseline, a

Figure 10: Confusion matrix for each direction 15

majority class classifier (guessing the most frequent class) which performed 22.9%. Fig.10 shows the confusion matrix, clearly confirming these points.

This result is useful even though it is negative; it indicates quantitatively that the sensor alone cannot support directional gestures. For example, it cannot distinguish between a "swipe left" gesture and a "swipe right" gesture. This motivates a change in our gesture dictionary, towards gesture based on distance rather than on directions.

9.4 Height and Distance Experiment

Since deducing that directions cannot be distinguished from one another, the next step was treating the raw data reading as a function of the other variables: the horizontal (reach) and vertical (height) distance from the sensor, as well as the distance (regardless of height or reach).

To model this relationship, we considered an exponential decay model of distance. However, the influence of height ended up being important as well, and this was something a single variable model wouldn't be able to capture. That's why we fitted a model combining both effects:

$$\text{raw_value} = \frac{A}{\text{distance} + B} + C \cdot \text{height} + D \quad (6)$$

In this model, the distance is inversed to reflect the expected fall of the sensing capability with distance. B is an offset preventing dividing by 0 in case of no distance. C stands for the influence of the height. And D adds the constant baseline in the reading.

We used the dataset to fit this model by non-linear least squares, getting the following results, with $R^2 = 0.6675$:

- $A = 8613.26$
- $B = 1.98$
- $C = 14.15$
- $D = -537.71$

For the "above" direction, we examined the raw data as a function of height alone (as the horizontal distance was none. As height increases, the signal decreases, and this stays consistent with the behavior or the fitted model.

9.5 Implications of Gesture Design

The Bravo sensor experiments led to changes in our gesture dictionary, following on our conclusion that we couldn't use directional gestures. This ruled out any gesture which meaning depends on direction, such as pointing or swiping in a certain direction. We thus redefined the gesture dictionary around gestures that could be measured by the sensor: gestures based on presence and distance rather than direction.

That way, the swipe gesture cannot be directional (a swipe left or right for example), but rather a unique gesture with one single meaning, detectable by the hand coming closer, then being present on top of the robot, and then going away, at a constant rate.

Adding time as a parameter allows for more diversity in a restricted gesture dictionary. Indeed, the hand staying on top of the robot for a specific amount of time could mean a select gesture. Without the notion of time, this gesture could have been detected as a swipe.

Overall, the results from the Bravo hand sensor experiments narrowed the feasibility of the previously brainstormed gestures to gestures built on presence vs. absence, distance, and time.

10 Results and Validation

10.1 Simulator Validation

The simulator was first tested independently of the user interface to verify the main components of the simulation engine. The tests were run directly on `WorldEngine`, `DifferentialDriveModel`, `IRSensorEngine`, and `VirtualNetwork`, using scripted scenarios with fixed inputs and a fixed number of simulation ticks.

Movement For straight-line motion, a robot was commanded to drive both wheels at equal speed for two seconds. With a motor input of 0.5, the differential drive model predicts a displacement of 0.15 m. The simulated displacement was 0.125 m. The difference of 0.025 m was caused by a collision with a stationary robot located 0.15 m in front of the moving robot. The collision-resolution mechanism therefore stopped the robot before it reached the predicted position. This test cannot be used as an isolated measurement of the motion model, an obstructed scenario is required to evaluate the kinematic model independently of collision handling.

Rotation For rotation, opposite velocities were applied to the two wheels for two seconds. The expected change in heading was 0.5625 rad, which matched the simulated value exactly. No measurable error was observed in this test.

Infrared Detection Range The IR sensing model was tested by placing a hand at horizontal distances ranging from 0 to 20 cm from a robot. The hand was detected at 5 cm, but not at 0 cm or between 10 and 20 cm, despite the configured 15 cm passive detection range.

Two effects explain these results. At 0 cm, the hand is directly above the robot and has no horizontal bearing, which causes the current sensor model to reject the detection. At larger distances, another robot in the scene can produce a stronger normalized signal because active-robot detection uses a larger range than passive hand detection. Since the sensor reports only its strongest candidate, the robot signal can therefore suppress the hand signal.

The test shows that detection in the current model depends not only on the distance to the hand, but also on the other subjects simultaneously visible to the sensor.

Message Transmission Twenty broadcast messages were sent from one robot to the three other robots in the swarm. All recipients received all twenty messages. Under the current network model, this confirms lossless message transmission. Packet loss, communication range, and latency are not simulated and therefore cannot be evaluated by this test.

Repeatability Finally, the same scripted scenario was executed twice from identical initial conditions. Both runs produced the same final robot state, ($x = 0.089474$, $y = -0.008413$, $\Theta = -0.1875$). The simulator is therefore deterministic under these conditions.

10.2 Collective Behavior Results

Two collective motion behaviors were evaluated through the C++ runtime pipeline on four robots: a Vicsek-style heading-alignment behavior and a Reynolds-style flocking behavior.

Vicsek Heading Alignment The heading-alignment behavior was evaluated in two trials with different initial conditions. In the first trial, the robots started with similar headings and remained within approximately 0.02 rad ($\approx 1^\circ$) of one another throughout the observed period. The resulting order parameter was $r \approx 1.00$, indicating near-perfect alignment.

In the second trial, the robots started with substantially different headings. At the observed instant, three robots had converged to similar headings (0.302, 0.297, and 0.300 rad), while the fourth robot had a heading of -2.836 rad, approximately π radians from the other three. The resulting order parameter was $r \approx 0.50$, corresponding to three robots aligned in one direction and one in the opposite direction.

These two trials show that the local heading-averaging rule can produce full alignment under some initial conditions, but does not guarantee global consensus for arbitrary initial headings. The second trial also illustrates a limitation of the averaging rule: locally consistent headings can result in a partially aligned configuration rather than a single common direction.

Reynolds Flocking A second experiment evaluated the combination of separation, cohesion and alignment using the Reynolds flocking rules. The behavior was executed through the same C++ runtime pipeline. At startup, each robot perceived between two and three neighbors and computed an initial motion target consisting of a forward displacement of approximately 10 - 30 mm with a small heading correction. This confirmed that the local perception and weighting mechanisms were producing motion commands from the beginning of the simulation.

After approximately 33 seconds of continuous operation, the swarm reached a mean inter-robot distance of 143.8 mm. This value remained within the range defined by the separation radius (120 mm) and cohesion radius (400 mm), indicating that the robots maintained a coherent group without becoming excessively close or dispersing. At the same time, the heading order parameter reached $r = 0.998$, indicating that the four robots were moving in nearly the same direction.

The experiment therefore shows that the combination of the three local flocking rules can produce both spatial cohesion and strong directional alignment in the tested configuration. However, the reported values correspond to the observed state after approximately 33 seconds; a more detailed time-series analysis would be needed to characterize how the swarm transitions from its initial configuration to this state.

10.3 Gesture Recognition results

The gesture recognition pipeline was tested functionally in the simulator to verify that hand trajectories could be recorded, processed by the \$1 recognizer, and associated with the corresponding gesture labels. The system successfully recognized the gesture included in the implemented vocabulary and returned a similarity score for each recognized trajectory.

However, no systematic quantitative evaluation of recognition accuracy was conducted during the internship. In particular, the number of test samples was not sufficient to compute reliable accuracy or per-gesture performance, and no controlled evaluation of the influence of gesture speed or hand position was performed. The main role of the recognition experiments was therefore to validate the integration of trajectory recording and gesture classification within the simulator. A more complete evaluation would require a larger dataset containing multiple executions of each gesture, performed at different speeds and positions and evaluated separately on real and perceived trajectories.

10.4 Bravo Sensor Results

The Bravo sensor experiments were used to determine which types of hand gestures could realistically be supported by the available sensor. Two main results emerged.

First, the sensor did not provide a reliable signal for determining the horizontal direction of a gesture. A KNN classifier trained on the raw sensor values achieved 21.8% accuracy, compared with 22.9% for the majority-class baseline. The raw sensor reading therefore did not contain a usable directional signal in the tested conditions.

Second, the sensor reading depended on both the horizontal distance of the hand and its height above the sensor. A combined model,

$$\text{raw-value} = \frac{A}{\text{distance} + B} + C \cdot \text{height} + D, \quad (7)$$

provided a better fit to the collected data than the distance-only exponential model, with $R^2 = 0.6675$.

These results directly affected the gesture vocabulary used in the simulator. Directional gestures such as swipe-left and swipe-right were not retained as gestures that could be reliably inferred from the sensor alone. Instead, the vocabulary focuses on properties that the sensor can measure more directly, such as presence, distance, and dwell time.

11 Discussion and Limitations

11.1 Interpretation of Results

The results show that the proposed approach is feasible as a framework for human-swarm interaction, but also reveal several constraints that have to be considered when designing the interaction. The simula-

tor successfully brings together local robot perception, inter-robot communication, collective motion and trajectory-based gesture recognition in a single environment. This makes it possible to study the interaction between these components rather than evaluating them independently.

The collective behavior experiments also show that local rules can produce coordinated motion without requiring a centralized controller. However, the alignment experiment illustrates that local rules do not necessarily lead to a single global configuration. Depending on the initial conditions, the robots can converge to different stable configurations. This is particularly important for interaction: a command given to a swarm does not automatically result in a unique collective response simply because the underlying robots follow the same local rule.

The gesture experiment provide a similar distinction between what can be recognized and what can actually be perceived by the robots. The $\$1$ recognizer provides a simple way of classifying hand trajectories once a sufficiently informative trajectory has been obtained. However, the Bravo sensor experiments show that the available local sensing does not provide enough information to recover every property of a gesture. In particular, horizontal direction could not be reliably inferred from the sensor measurements. Gesture design therefore cannot be separated from the capabilities of the sensing hardware.

This also explains the importance of maintaining separate real and perceived trajectories in the simulator. Recognition from the real trajectory measures the capability of the recognition method itself, while recognition from the perceived trajectory also includes the errors introduced by the robots' local observations. The two measurements therefore answer different questions and should not be treated as interchangeable.

Overall, the results suggest that a practical gesture vocabulary for a decentralized swarm has to be designed jointly with the sensing and collective-control mechanisms. Gestures that are natural for a human are not necessarily gestures that can be reliably distinguished from local robot observations. Conversely, relatively simple signals such as presence, distance, or dwell time can provide useful interaction information when they match the capabilities of the sensors.

11.2 Gesture Feasibility

The sensor characterization places direct constraints on the gesture vocabulary. Gesture based on the presence or proximity of the hand are compatible with the available sensing, whereas gestures that require reliable estimation of horizontal direction are more difficult to support. This distinction is important because a gesture may be easy to recognize from a recorded trajectory while remaining impossible to identify from the information available to an individual robot.

The spatial relationship between the hand and the robot also affects whether a gesture can be detected. The sensor response depends on both horizontal distance and height, and the experiments showed that the hand is not detected uniformly throughout the nominal sensing range. In addition, the presence of other subjects can affect which signal is reported by the sensor. As a result, gestures cannot be defined independently of where they are performed relative to the robots.

Gesture similarity is another constraint. Gestures whose trajectories are very similar can be difficult to distinguish using a template-based recognizer, particularly when the available number of templates is small. The gesture vocabulary should therefore favor trajectories that are sufficiently distinct from one another and that can be performed consistently by the user.

These observations suggest that gesture design should be driven by both recognition and perception. A gesture should not only be visually or physically intuitive, it should also produce a sufficiently distinct and reproducible signal for the robots to interpret it reliably.

11.3 Gesture Recognition Limitations

The trajectory-based recognition approach also has several limitations. The current gesture vocabulary contains a relatively small number of templates, which limits the range of interactions that can be represented. Adding more gestures would increase the expressive capacity of the system, but could also introduce confusion between trajectories with similar shapes.

The $\$1$ recognizer is based on the spatial trajectory of the gesture and does not explicitly use all of the information available in the recorded trace, such as its timing or the orientation of the hand. This means that gestures with similar spatial trajectories but different execution dynamics may be treated as the same

gesture.

Recognition also depends on the quality of the reconstructed trajectory. Errors in local sensing can alter the perceived trajectory before it reaches the recognizer, making it difficult to distinguish errors caused by gesture classification from errors caused by perception. The comparison between real and perceived trajectories therefore remains an important part of evaluating the system.

Finally, the current evaluation is limited to the tested gestures and experimental conditions. The results should therefore not be interpreted as a general evaluation of trajectory-based gesture recognition. A larger dataset, more users, and a wider range of execution conditions would be required to assess the robustness of the approach more extensively.

11.4 Decentralized Interpretation

The system was designed with the decentralized nature of the swarm in mind. Individual robots provide local observations, and these observations can be combined through communication to obtain information about the hand and the ongoing interaction. This provides a basis for moving from individual observations to a collective interpretation.

In the current prototype, however, some parts of this process are still handled at a global level. The perceived hand trajectory is reconstructed by combining observations from several robots, and the trajectory-based gesture recognition is performed on this reconstructed trajectory. The mapping between recognized gestures and robot actions is also currently handled at the simulator level.

These choices were appropriate for the scope of the internship and allowed the different components of the interaction to be developed and evaluated independently. A natural continuation of the work would be to distribute more of the interpretation process across the robots, so that gesture recognition and command decisions can be performed using only information available locally or exchanged between neighboring robots.

11.5 Simulator Limitations

The simulator provides a controlled environment for developing the interaction system but it necessarily simplifies several aspects of the physical platform. The robot motion model does not reproduce all physical effects that may influence the real robots, such as wheel slip or differences in motor responses. Similarly, the infrared sensing model is an approximation of the physical sensors and cannot reproduce all sources of sensor noise and variability.

The communication model is also simplified. Messages are delivered reliably in the current simulator, without packet loss, communication delays, or range limitations. The results obtained for communication therefore only validate the behavior of the simulated network and cannot be used to predict communication performance on the physical robots.

Timing is another difference between the two environments. The simulator advances the robot behaviors in discrete simulation steps, whereas the physical robots operate with real sensor, communication and actuator timing. These differences may affect the behavior of systems that depend on precise timing or rapidly changing sensor measurements.

Finally, the simulator does not model effects such as battery depletion. The behaviors observed in simulation should therefore be interpreted as validation of the implemented control and interaction logic under the simulated conditions, rather than as a direct prediction of physical performance.

11.6 Physical Platform Limitations

The validation of the proposed interaction is also limited by the availability of the physical platform. The number of robots available for testing was limited, and the physical swarm could not be used for the complete set of experiments performed in simulation.

The Bravo sensor experiments nevertheless provided information about an important part of the physical interaction problem. In particular, they showed that sensor measurements are affected by the position of the hand and do not provide reliable directional information under the tested conditions. However, these experiments do not capture all of the variability that may occur when several robots operate simultaneously, communicate and move in a physical environment.

Consequently, the results obtained in simulation should not be interpreted as a complete validation on the physical swarm. Physical experiments with a larger number of robots would be required to evaluate the complete interaction pipeline under realistic sensing, communication and timing conditions.

12 Future Work

12.1 Deployment on Physical Robots

The first step in the continuation of this work is the transfer of behaviors and gesture recognition mechanisms developed in simulation to the physical Octon robots. This will validate whether the local-perception constraints, flocking behavior, and gesture-driven interaction designed and tested here hold under real sensor noise, actuation limits, and communication conditions that the simulator can only approximate, and will surface any assumptions made during development that do not transfer cleanly to hardware.

12.2 Integration of the real Robot Firmware

The simulator should eventually compile and execute the same C++ code that runs on the physical robots, rather than a SimHAL-specific version of it. This would close the gap between what is validated in simulation and what actually ships to hardware, removing the current dependency on manually keeping the two codebases consistent and ensuring that a behavior confirmed in the simulator is guaranteed to behave identically once deployed.

12.3 Continuation of the Hand Sensor Experiment

An amelioration of the hand sensor experimental setup could include an automated object (two rails, for example) placing objects at different distances from the sensor in complement to the script. This would allow for easier repeatability and a better accuracy regarding positioning.

The experiment could also benefit from more data and analysis of different hand setups (eg. closed or open hand, etc), in order to quantify the impact of these different setups. As the script is already using this information as a parameter, all previous data can be used for this means as well, and the script can be used to record new data without needing to be changed.

12.4 Improved Infrared Sensor Model

The current simulated infrared perception model could be refined using empirical data collected from the physical Bravo robots, replacing the simplified geometric cone and intensity model used here with one calibrated against real sensor readings. This would narrow the reality gap between simulated and physical IR perception, making simulation results a more reliable predictor of on-hardware behavior.

12.5 More Robust Gesture Recognition

The gesture recognition pipeline would benefit from a larger and more diverse dataset: more users, more gesture samples per gesture, and a larger overall vocabulary than the small set tested here. With more data available, statistical approaches such as Hidden Markov Models or other machine learning methods could be explored as alternatives or complements to the trajectory-based recognizer used in this work, potentially improving robustness to the kind of user variability that a template-based method like \$1 is inherently more sensitive to.

12.6 Multi-user Interaction

The system currently assumes a single user interacting with the swarm at a time. Extending it to support multiple simultaneous users, each controlling a distinct group of robots, would require resolving how the swarm distinguishes between different hands, how robots decide which user's commands apply to them, and

how the local perception constraint that governs single-user interaction should be adapted so that multiple concurrent gesture streams do not interfere with one another.

12.7 Extension of the Octon Puzzle Concept

Beyond the interaction mechanisms studied here, the underlying Octon Puzzle concept could be developed into a complete experimental application. This would include several difficulty levels, a set of formation cards defining target shapes for the swarm to reproduce, rules supporting multiplayer play, a scoring mechanism, and automatic evaluation of how closely the swarm’s resulting formation matches the target, turning the current research into a self-contained interactive experience.

13 Conclusion

Our initial problem was to make a decentralized swarm able to be directed by hand gestures in a unified way. Because no robot has a global view of the swarm, decentralization makes swarms hard to direct, even though it makes it more robust in other ways. We explored how natural hand gestures could be translated into coherent and coordinated collective behaviors in the swarm. We made three contributions: an Octon simulator modeling human swarm interaction, flocking behavior and gesture interaction, an analysis of the Bravo hand sensor, and the investigation of a gesture vocabulary that would translate well to the swarm, using context.

Our results show the possibilities of our work, while highlighting areas of improvement and the need to complete certain parts that were not doable due to the short length of the internship. Our simulator behaves as intended, as shown in the validation part: rotation matches the model, and messages are delivered reliably. However, we highlight a limitation of the IR model, where detection depends on which other objects are simultaneously in view, and not only distance to the object. Our collective behavior experiment showed that a local heading alignment drove the four robots to a shared heading reliably if there was a favorable starting condition, but with different initial headings it wasn’t guaranteed. This shows that local rules do not guarantee the same outcome. With Reynolds flocking, we obtained better results, observing strong alignment and cohesion. Looking at sensor experiments, we have determined that horizontal decision could not be recovered from the raw values (this was shown by a 21.8% KNN accuracy against a 22.9% baseline) but that distance and height could be used to explain that values. From this, we narrowed the gesture vocabulary, from many brainstormed concepts to gestures mostly built around time, presence and distance.

To answer our research question ("how can natural hand gestures be interpreted and translated into coherent, coordinated, collective behaviors in a decentralized robot swarm?"), we have found that the interpretation of natural hand gestures can be interpreted and used by a decentralized swarm, when the gesture vocabulary, its recognition and its interpretation are built around the swarm’s limitations. Indeed, presence, distance and time enable selection and the tuning of flocking behaviors, while we cannot use direction with the current sensors. We were able to simulate these interactions, but without experimentation on physical robots, it remains hypothetical. Deployment on physical robots is thus the next step for closing the gap.

References

- [1] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, “Swarm robotics: A review from the swarm engineering perspective,” *Swarm Intelligence*, vol. 7, no. 1, pp. 1–41, 2013. DOI: 10.1007/s11721-012-0075-2
- [2] A. Kolling, S. Nunnally, and M. Lewis, “Towards human control of robot swarms,” in *Proceedings of the Seventh Annual ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, New York: ACM, 2012, pp. 89–96.
- [3] G. Podevijn, R. O’Grady, and M. Dorigo, “Self-organised feedback in human swarm interaction,” in *Proceedings of the Workshop on Robot Feedback in Human-Robot Interaction: How to Make a Robot Readable for a Human Interaction Partner (Ro-Man)*, 2012.

- [4] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 25–34, 1987. DOI: 10.1145/37402.37406
- [5] T. Vicsek, A. Czirók, E. Ben-Jacob, I. Cohen, and O. Shochet, "Novel type of phase transition in a system of self-driven particles," *Physical Review Letters*, vol. 75, no. 6, pp. 1226–1229, 1995. DOI: 10.1103/PhysRevLett.75.1226
- [6] A. E. Turgut, H. Çelikkanat, F. Gökçe, and E. Şahin, "Self-organized flocking in mobile robot swarms," *Swarm Intelligence*, vol. 2, no. 2-4, pp. 97–120, 2008.
- [7] A. Giusti, J. Nagi, L. Gambardella, and G. Di Caro, "Distributed consensus for interaction between humans and mobile robot swarms," in *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, Richland, SC, 2012.
- [8] J. O. Wobbrock, A. D. Wilson, and Y. Li, "Gestures without libraries, toolkits or training: A \$1 recognizer for user interface prototypes," in *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2007, pp. 159–168. DOI: 10.1145/1294211.1294238
- [9] R.-D. Vatavu, L. Anthony, and J. O. Wobbrock, "Gestures as point clouds: A \$p recognizer for user interface prototypes," in *Proceedings of the 14th ACM International Conference on Multimodal Interaction (ICMI)*, 2012. [Online]. Available: <https://faculty.washington.edu/wobbrock/pubs/icmi-12.pdf>
- [10] S. Kratz and M. Rohs, "A \$3 gesture recognizer: Simple gesture recognition for devices equipped with 3D acceleration sensors," in *Proceedings of the 15th International Conference on Intelligent User Interfaces (IUI)*, 2010, pp. 341–344. DOI: 10.1145/1719970.1720026
- [11] M. Le Goc, L. H. Kim, A. Parsaei, J.-D. Fekete, P. Dragicevic, and S. Follmer, "Zoids: Building blocks for swarm user interfaces," in *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (UIST)*, 2016, pp. 97–109. DOI: 10.1145/2984511.2984547
- [12] L. H. Kim, D. S. Drew, V. Domova, and S. Follmer, "User-defined swarm robot control," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 2020, pp. 1–13. DOI: 10.1145/3313831.3376814